

Pipeline RAG

Esther Menéndez, Ema Holinaty, Guillermo García

April 2026

1. Introducción y objetivos

En este proyecto se ha construido un pipeline RAG cuyo objetivo es responder preguntas sobre un tema específico empleando abstracts previamente descargados sobre dicho tema. Un pipeline es una secuencia estructurada de operaciones donde la salida de un paso es la entrada del siguiente. En este caso, al ser un RAG (*Retrieval Argumented Generation*) no es simplemente un modelo de lenguaje (LLM) que genera respuestas en función del entrenamiento recibido sino que busca documentos relevantes (en este caso los abstracts guardados), los añade al prompt y genera una respuesta basándose en esa información. Es decir, el modelo primero consulta una base documental y luego responde.

2. Código

El código está estructurado en diferentes celdas que ejecutan cada fase del proceso: descarga de librerías, obtención de abstracts, chunking y embedding, pregunta y respuesta, así como una celda adicional para obtener una gráfica UMAP con la pregunta y contexto recuperado. Esto no solo permite supervisar que cada sección funciona correctamente antes de saltar a la siguiente si no que también optimiza el proceso de generación de respuesta al no tener que descargar los abstracts y crear el índice con los embeddings cada vez que se hace una pregunta.

2.1. Librerías

Las librerías empleadas a lo largo del código son:

- **arxiv**: consulta de la API de Arxiv.

- **re**: limpieza del texto
- **pandas**: manejo de datos tabulares
- **langchain-text-splitters**: divide el texto en chunks
- **sentence-transformers**: genera embeddings
- **numpy**: manejo de arrays numéricos
- **faiss**: motor de búsqueda vectorial
- **CrossEncoder**: modelo de reranking
- **transformers**: carga modelos de lenguaje
- **langchain-core**: construye el prompt dinámico y envuelve la función del LLM como pipeline

2.2. Recopilación de abstracts

El primer paso del pipeline es la descarga de los abstracts que serán empleados como contexto más adelante. El tema específico que ha sido seleccionado es *open cluster AND Gaia*. Una vez descargados los abstracts desde Arxiv, se realiza una limpieza mínima que quita los saltos de línea y los espacios duplicados. No es posible una limpieza más exhaustiva ya que al tratarse de un abstract, se perdería información como palabras específicas del tema, por ejemplo. Después, se crea con pandas un fichero *.csv* en el que se guardan los abstracts junto con el link al paper en arxiv, el título del paper y el año de publicación.

2.3. Chunking y embedding

Una vez guardados los abstracts, se procede a su división en fragmentos de menor tamaño (*chunks*) para su posterior procesamiento. Esto es el llamado *chunking* cuyo tamaño es de 800 caracteres y tienen un overlap de 100. Una vez habiendo fragmentado los abstracts, se crea un nuevo fichero *.csv* donde se guarda un índice para poder acceder a cada chunk, el propio texto fragmentado, una vez más el título y el año del paper y, por último, el texto original del que ha salido el chunk, es decir, el abstract completo. El código correspondiente sería el siguiente:

```
# CHUNKING
splitter = RecursiveCharacterTextSplitter(
    chunk_size=800,
    chunk_overlap=100 )

all_chunks = []
for i, (_, row) in enumerate(df.iterrows()):
    text = row["title"] + ". " + row["abstract"]
    chunks = splitter.split_text(text)

    for j, chunk in enumerate(chunks):
        all_chunks.append({
            "chunk_id": len(all_chunks),
            "text": chunk,
            "title": row["title"],
            "year": row["year"],
            "full_text": text,
            "id": row["id"]
        })

chunk_df = pd.DataFrame(all_chunks)
chunk_df = chunk_df.reset_index(drop=True)

print(f"Total chunks: {len(chunk_df)}")
```

A continuación es necesario hacer el embedding. El proceso de embedding consiste en hacer pasar frases a vectores, en este caso convertir cada chunk en un vector empleando el modelo "BAAI/bge-base-en-v1.5". Para ello, en primer lugar divide las palabras, las tokeniza. Una vez divididas en subpalabras, asigna a cada una un vector numérico que corresponde a una especie de diccionario aprendido. Posteriormente,

entra en juego el transformer que mira todas las palabras a la vez (*self-attention*) dotando a cada token de significado contextual. Es decir, ahora cada vector correspondiente a un token depende de toda la frase. El siguiente paso es el *pooling*: tenemos un vector por cada token pero necesitamos uno solo. Para ello, se promedian todos los vectores obteniendo un único vector final correspondiente al significado del chunk. Por último, se normalizan todos los vectores de forma que sea posible emplear *cosine similarity* más adelante. Todos los pasos necesarios se resumen en las siguientes líneas de código:

```
# EMBEDDINGS
model = SentenceTransformer('BAAI/bge-base-en-v1.5')

texts = chunk_df["text"].tolist()

embeddings = model.encode(
    texts,
    normalize_embeddings=True,
    show_progress_bar=True
```

2.4. Índice faiss

Puesto que más adelante va a ser necesario tener los embeddings clasificados para buscar el que responda a la pregunta formulada, se ha usado FAISS para crear el índice al que acceder más adelante. En el código se encuentra implementado de la siguiente forma:

```
# FAISS
dimension = embeddings.shape[1]

index = faiss.IndexHNSWFlat(dimension, 32)
index.metric_type = faiss.METRIC_INNER_PRODUCT

index.add(embeddings)

print(f"Índice FAISS creado con {index.ntotal} vectores")

# Guardar índice FAISS
faiss.write_index(index, "faiss_index.bin")
```

Uno de los aspectos clave a la hora de estructurar el índice faiss es elegir el tipo de búsqueda que se va a seguir más adelante, en este caso HNSW (*Hierarchical Navigable Small World*). De esta forma se construye un grafo inteligente que en vez de buscar comparan-

do todo (un método muy lento), lo hace comparando con los vecinos cercanos. Es decir, considerando cada embedding como un nodo, cada nodo se conecta con los cercanos (semánticamente similares) con 32 conexiones por nodo. De esta forma, a la hora de buscar se comienza por un nodo cualquiera, se navega por el grafo y la búsqueda se va acercando al vector más parecido.

La métrica de similitud empleada es el producto escalar. Sin embargo, puesto que durante el embedding los vectores fueron renormalizados, el producto escalar es equivalente al *cosine similarity*.

2.5. Pregunta y respuesta

En este bloque se aborda la generación de respuestas dentro del sistema. Antes de ello, es importante introducir **LangChain**, un framework de código abierto diseñado para la construcción de aplicaciones basadas en modelos de lenguaje. Su API es de gran utilidad para encadenar distintos componentes como prompts, modelos de lenguaje o bases de datos vectoriales, facilitando la creación de pipelines modulares.

2.5.1. Elección del LLM

Uno de los aspectos más relevantes de esta sección es la elección del modelo de lenguaje (LLM). Esta decisión ha estado fuertemente condicionada por las limitaciones de la GPU disponible en Google Colab, que es un entorno compartido y temporal. Habitualmente, se asigna una GPU T4 con aproximadamente 16 GB de VRAM, aunque esto depende de la disponibilidad en cada sesión. Por este motivo, se ha optado por el modelo **FLAN-T5-Large**, desarrollado por Google, ya que su menor número de parámetros (770 millones respecto a los 7.000 millones de Mistral 7B) y su diseño arquitectónico lo hacen significativamente más eficiente en entornos con recursos limitados. Esto permite su ejecución sin saturar la memoria de la GPU, aunque es a costa de una menor capacidad de generación y razonamiento en comparación con modelos de mayor escala.

FLAN-T5-Large se basa en el modelo T5 (Text-To-Text Transfer Transformer), que unifica las tareas de procesamiento de lenguaje natural como transfor-

maciones de texto de entrada a texto de salida. Este enfoque permite emplear una única arquitectura para tareas como traducción, resumen o respuesta a preguntas, simplificando tanto el entrenamiento como su aplicación.

Además, el modelo ha sido afinado mediante *instruction tuning*, técnica que lo entrena con instrucciones en lenguaje natural y sus correspondientes respuestas. Esto mejora su capacidad para interpretar la intención del usuario y seguir indicaciones de forma más directa.

A nivel arquitectónico, **FLAN-T5-Large** sigue un esquema encoder-decoder. El encoder procesa el input completo (contexto y pregunta) y genera una representación semántica, que el decoder utiliza para producir la respuesta de forma autoregresiva. Esto lo diferencia de modelos decoder-only como GPT o Mistral, que reprocesan el contexto en cada paso de generación, aumentando el coste computacional pero ofreciendo mayor flexibilidad. Asimismo, el modelo presenta en su diseño un límite de 512 tokens en el input, lo que restringe la cantidad de contexto que puede utilizar para generar respuestas.

2.5.2. Librerías y carga del modelo

```
import faiss
import numpy as np

from sentence_transformers import CrossEncoder
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

from langchain_core.prompts import PromptTemplate
from langchain_core.runnables import RunnableLambda

# LLM
model_name = "google/flan-t5-large"

tokenizer = AutoTokenizer.from_pretrained(model_name)
model_llm = AutoModelForSeq2SeqLM.from_pretrained(model_name)
```

Tras importar las librerías se carga el modelo de lenguaje y, utilizando las clases **AutoTokenizer** y **AutoModelForSeq2SeqLM** de transformers, se crean los objetos **tokenizer** y **model_llm** que se utilizarán más adelante.

- **AutoTokenizer:** Convierte texto a tokens para que el modelo pueda leerlo en la parte de encoder y volver a pasarlo de token a texto en el decoder.

- **AutoModelForSeq2SeqLM:** Es la clase para cargar el modelo. Elige automáticamente la arquitectura correcta según el modelo que se ha elegido, que en este caso es un Seq2Seq (encoder-decoder).

2.5.3. El prompt

Definimos el prompt, que traerá como inputs el contexto y la pregunta a realizar al LLM.

```
# PROMPT
prompt = PromptTemplate(
    input_variables=["context", "question"],
    template="""
You are a scientific assistant.

Use the context to answer the question.
You must rephrase and summarize the information.

If the answer is not present, say "Not enough information".

Context:
{context}

Question:
{question}

Answer in your own words:
""")
```

La formulación del prompt tiene un impacto significativo en la calidad de la respuesta generada. Debido al entrenamiento mediante instruction tuning de FLAN-T5 las instrucciones más precisas y bien definidas son de especial importancia. Además, en este caso resulta fundamental indicar explícitamente que el modelo utilice únicamente la información contenida en el contexto proporcionado, con el objetivo de reducir el riesgo de alucinaciones o generación de contenido no fundamentado. Además, es recomendable incluir instrucciones como “responder con sus propias palabras”, ya que esto favorece la reformulación de la información y evita la reproducción literal del contexto, algo que modelos con menos parámetros como este pueden tender a hacer, especialmente con un contexto reducido.

2.5.4. Función de Generación y Reranker

```
# GENERATION FUNCTION
def llm_generate(inputs):
    prompt_text = prompt.format(**inputs)

    tokens = tokenizer(
        prompt_text,
        return_tensors="pt",
        truncation=True,
        max_length=512
    )

    outputs = model_llm.generate(
        **tokens,
        max_new_tokens=150
    )

    return tokenizer.decode(outputs[0], skip_special_tokens=True)

rag_chain = RunnableLambda(llm_generate)

# RERANKER
reranker = CrossEncoder('cross-encoder/ms-marco-MiniLM-L-6-v2')
```

La función de generación `llm_generate` es el componente encargado de transformar el contexto recuperado y la pregunta del usuario en una respuesta en lenguaje natural con FLAN-T5. En primer lugar, el texto del prompt se procesa con el `tokenizer` que se ha creado antes, que lo convierte en una secuencia de tokens compatible con el modelo. Durante este paso se aplica truncamiento con un límite de 512 tokens, lo cual es necesario para ajustarse a la capacidad máxima del modelo, aunque puede implicar la pérdida de parte del contexto si este es demasiado extenso. Posteriormente, el modelo genera la respuesta de manera autoregresiva (`model_llm`), produciendo nuevos tokens hasta un máximo de 150, lo que permite controlar la longitud de la salida. Finalmente, estos tokens se decodifican a texto legible.

`RunnableLambda` se utiliza para convertir la función `llm_generate` en un componente compatible con la arquitectura de procesamiento de LangChain. Esto se hace porque la función por sí sola es una función estándar de Python que recibe un diccionario con el contexto y la pregunta, y devuelve una respuesta generada por el modelo, pero, para integrarla dentro de un pipeline más estructurado, LangChain necesita que dicha función se represente como un objeto ejecutable dentro de su sistema de cadenas (chains). De esta manera más adelante se podrá invocar a la función manteniendo compatibilidad con otros componentes del framework.

Por otro lado, reranker se crea utilizando la clase `CrossEncoder` de sentence-transformers, y está basado en `Cross-Encoder MiniLM`. Aunque FAISS permite recuperar rápidamente los chunks más cercanos en el espacio vectorial, esta similitud se basa únicamente en embeddings y puede no reflejar completamente la relevancia semántica real respecto a la pregunta. Para refinar estos resultados, el `reranker` evalúa directamente pares formados por la pregunta y cada fragmento de texto, procesándolos conjuntamente en un único modelo (MiniLM) que tiene en cuenta la interacción completa entre ambos. Esto permite obtener una puntuación de relevancia más precisa. A partir de estas puntuaciones, los fragmentos se reordenan y se seleccionan los más relevantes para construir el contexto final que se proporcionará al modelo generativo.

2.5.5. Definición del Pipeline

Finalmente definimos la función que ejecutará el pipeline, esta implementa el núcleo del sistema de Retrieval-Augmented Generation.

```
# RAG PIPELINE
def rag(question):

    # ----- EMBEDDING -----
    question_emb = model.encode([question],
                                normalize_embeddings=True
                                ).astype("float32")
```

- **EMBEDDING** En primer lugar, la pregunta del usuario se convierte en un embedding vectorial mediante el modelo definido en la celda de Chunking y Embedding (`model = SentenceTransformer('BAAI/bge-base-en-v1.5')`). Este embedding constituye una representación numérica del significado de la pregunta y permite realizar búsquedas eficientes en el espacio vectorial. Para garantizar la compatibilidad con la base de datos vectorial, el embedding se normaliza y se convierte al formato adecuado.

```
# ----- RETRIEVAL (FAISS) -----
D, I = index.search(question_emb, 5)

retrieved = []
print("\n=== FAISS RESULTS ===\n")

for score, i in zip(D[0], I[0]):
    item = chunk_df.iloc[i]

    print(f"FAISS score: {score:.4f} | {item['title']}")

    retrieved.append({
        "text": item["text"],
        "title": item["title"],
        "year": item["year"],
        "faiss_score": score,
        "id": item["id"] })
```

- **RETRIEVAL (FAISS)** A continuación la pregunta del usuario, previamente transformada en un embedding (`question_emb`), se utiliza como consulta en el índice vectorial que se ha creado antes.

La operación `index.search(question_emb, k)` permite recuperar los, en este caso, `k=5` vectores más cercanos al embedding de la pregunta, donde la cercanía se mide mediante una métrica de similitud (en este caso, producto interno, equivalente a la *similitud coseno* al trabajar con embeddings normalizados). Como resultado, FAISS devuelve dos elementos: por un lado, los valores de similitud (scores, `D`), que indican el grado de cercanía entre la pregunta y cada fragmento, y por otro, los índices `I` que permiten identificar qué fragmentos del corpus corresponden a esos vectores. Es importante destacar que esta búsqueda se realiza de forma extremadamente eficiente incluso en colecciones de gran tamaño, gracias a las estructuras de datos optimizadas que implementa FAISS. Sin embargo, la similitud utilizada en esta fase se basa únicamente en la proximidad en el espacio vectorial, lo que implica que los resultados recuperados son candidatos relevantes, pero no necesariamente los más adecuados desde un punto de vista semántico profundo. Por esta razón, esta etapa se complementa posteriormente con un proceso de reranking más preciso.

Creamos la lista de resultados `retrieved` y colocamos un `print` en esta parte del código para poder supervisar los scores de los chunks que se han recuperado.

A partir de los índices, se accede al DataFrame original para recuperar el contenido textual correspondiente y sus metadatos asociados, como el título del artículo, el año de publicación y el identificador del documento.

Estos elementos se agrupan en diccionarios que se añaden a la lista `retrieved`, de modo que cada entrada contiene tanto la información textual como la puntuación obtenida en la búsqueda (score).

```
# ----- RERANKING -----
pairs = [[question, c["text"]] for c in retrieved]
scores = reranker.predict(pairs)

ranked = sorted(
    zip(scores, retrieved),
    key=lambda x: x[0],
    reverse=True)

top_chunks = [c for _, c in ranked[:3]]
```

- **RERANKING** Como se ha mencionado, la fase de reranking tiene como objetivo refinar los resultados obtenidos en la etapa de recuperación con FAISS. Para ello se construye la variable `pairs`, que consiste en una lista de pares de entrada (pregunta y chunk de texto recuperado de `retrieved`). Cada elemento de esta lista combina la pregunta del usuario con uno de los fragmentos recuperados en la fase anterior. Esto permite que el modelo analice conjuntamente ambos elementos.

A continuación, CrossEncoder procesa estos pares y genera la variable `scores`, que contiene una puntuación numérica para cada par. Esta puntuación indica el grado de relevancia del fragmento de texto respecto a la pregunta, siendo mayor cuanto mejor responde el texto a la consulta.

Posteriormente, se construye la variable `ranked`, que consiste en una lista ordenada de los scores y chunks. De esta forma, los fragmentos más relevantes según el modelo quedan en las primeras posiciones.

Finalmente, se selecciona `top_chunks`, que corresponde a un subconjunto reducido de los me-

jores fragmentos de ranked (en este caso, los tres primeros). Estos fragmentos son los que se utilizarán para construir el contexto final que se introduce en el modelo generativo.

```
# ----- CONTEXT BUILDING -----
context = "\n\n".join([
    f"[Paper: {c['title']} ({c['year']})]\n{c['text'][:300]}"
    for c in top_chunks ])

print("\n=== CONTEXTO UTILIZADO ===\n")
print(context)
```

- **CONTEXT BUILDING** La fase de context building consiste en construir el texto final que se proporcionará al modelo generativo a partir de los fragmentos seleccionados en la etapa de reranking (`top_chunks`).

En primer lugar, se itera sobre cada elemento de `top_chunks`. Para cada uno de ellos, se extraen tanto el contenido textual como metadatos asociados, como el título del artículo y el año de publicación. Esta información adicional se incluye para aportar contexto y trazabilidad a la evidencia utilizada por el modelo.

El campo `'text'` de cada chunk se recorta a una longitud máxima (en este caso, 300 caracteres) con el objetivo de controlar que un solo chunk si es muy largo no ocupe el tamaño total del contexto.

Posteriormente, todos los fragmentos se concatenan en una única cadena de texto, separándolos mediante saltos de línea dobles. El resultado final es una representación estructurada del contexto, en la que cada fragmento aparece identificado por su paper de origen y acompañado de su contenido relevante.

```
# ----- GENERATION -----
return rag_chain.invoke({
    "context": context,
    "question": question })
```

- **GENERATION** Finalmente se invoca a la función de generación, que recibe como entrada el prompt formado por el contexto recuperado y la consulta original, para que genere la respuesta.

2.5.6. Generación de respuesta

```
# QUERY
question = "Insertar pregunta aquí"
response = rag(question)

print("\n=== RESPUESTA FINAL ===\n")
print(response)
```

Definimos la pregunta que queremos que el modelo responda y llamamos a la respuesta del RAG.

3. Resultados

Para determinar la viabilidad de nuestro sistema (RAG) en la extracción de información de literatura científica de física, se ha implementado un protocolo de evaluación dividido en tres dimensiones críticas: la capacidad de recuperación del motor vectorial, la fidelidad de la generación del lenguaje y la robustez del sistema ante consultas atípicas.

3.1. Evaluación del Componente de Recuperación (Retrieval)

El éxito de un sistema RAG depende de que el modelo consulte los documentos pertinentes. Para medir el desempeño del modelo en el *Retrieval*, nos basaremos tanto en los scores en la fase de recuperación FAISS, como cuánto se acercan los chunks finales al embedding de la pregunta que hemos realizado.

Una manera gráfica de observar este proceso de obtención de chunks es la representación en 2D del espacio de *embeddings*, ya que son vectores (aunque de 768D), utilizando la tecnología **UMAP** para reducir la dimensionalidad de éstos a una graficable manteniendo la estructura más importante de relaciones entre puntos.

Este modelo se aplica sobre los embeddings ya normalizados, permitiendo utilizar la métrica de similitud coseno (**metric=cosine**) para establecer las distancias 'semánticas' entre los puntos del gráfico. Los embeddings del corpus se proyectan a dos dimensiones mediante el método (**fit_transform**), como puntos azules. Para los chunks seleccionados por el mecanismo de retrieval hemos designado una cruz. El embedding de la pregunta se transforma al mismo espacio mediante (**transform**) y utilizando la misma

métrica, lo que asegura que la posición de la consulta sea coherente con la distribución de los documentos, y se puede distinguir por una estrella.

Se obtienen figuras como las que se muestran a continuación:

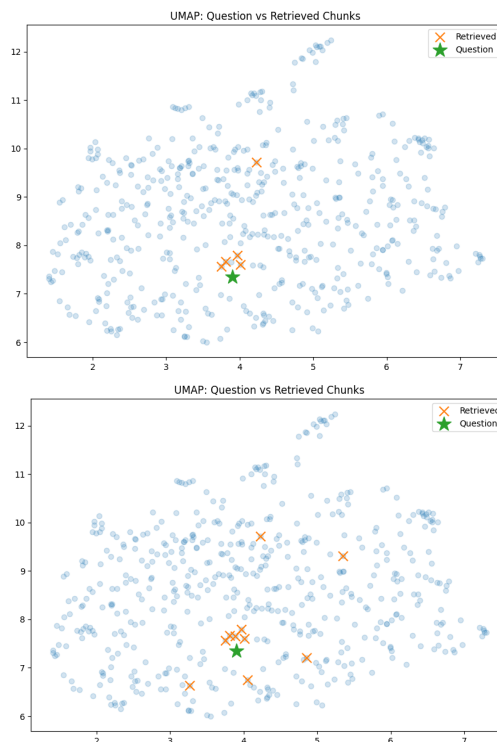


Figura 1: Ejemplo de gráficas UMAP. Comparación entre: (a) Arriba se limitó el retrieval a 5 chunks y (b) Aumentando los chunks de contexto se obtiene un rendimiento decreciente.

Algo que se puede observar fácilmente con esta representación es que aumentar el número de chunks de contexto no se traduce en una mejora apreciable en las respuestas emitidas, algo que medimos anteriormente. Llegamos así a la conclusión de que el mecanismo de *Retrieval* hace un buen trabajo seleccionando los chunks más relevantes para cada caso.

3.2. Evaluación de la Generación y Extracción de Datos

En esta fase, se midió la capacidad del modelo de lenguaje para sintetizar los abstracts recuperados sin introducir alucinaciones. Se analizaron manualmente las respuestas para determinar la precisión en la identificación de entidades técnicas. La manera de evaluar consistió en el planteamiento de preguntas de 3 niveles:

- **Nivel 1: Preguntas cuya respuesta está contenida en los abstracts:** Debido al buen desempeño de la fase de *Retrieval*, el modelo devuelve el texto respuesta a la pregunta de manera literal sin hacer grandes cambios.
- **Nivel 2: Preguntas sobre el tema, no contenidas de manera literal en el dataset:** Debido a las limitaciones del LLM, la generación de respuestas no innova demasiado y devuelve alguna frase relacionada de dentro del dataset. Por lo visto este LLM no parece confeccionar lenguaje desde cero sino que se limita a unir trozos de los abstracts.
- **Nivel 3: Peticiones de valores específicos:** Cuando se solicitan valores de magnitudes cuantitativas, se producen respuestas satisfactorias, de nuevo, devolviendo el fragmento de texto donde se encuentran.

Esto nos lleva a pensar que el modelo es incapaz de comprender el significado de las cuestiones discutidas y se basa en la similitud de los valores "semánticos" discutida en la fase de *embedding*.

3.3. Pruebas de Estrés

Para evaluar el anclaje del modelo a los datos adquiridos, hemos presentado una serie de preguntas sin relevancia sobre el tema e incluso lenguaje ilegible. En todos los casos el modelo se adherió a la instrucción de devolver "*Not enough information*".

3.4. Discusión sobre Limitaciones y Sesgos

A partir del seguimiento de los flujos de ejecución, se han identificado los siguientes cuellos de botella:

1. **Sensibilidad al fragmentado (Chunking):** Variar la longitud de los chunks es una manera de ajustar la cantidad de información que le aportamos al modelo. En este caso hemos observado que reducir demasiado la longitud de los chunks provoca respuestas incompletas y a veces incoherentes. Adicionalmente chunks más pequeños implica mayor cantidad de éstos y la velocidad de ejecución del código disminuye. Aumentar el tamaño en cambio no produce ninguna consecuencia notable puesto que el código devuelve de manera textual el fragmento con mayor similitud.
2. **Ingeniería de Prompts:** Debido a lo rudimentario de la selección de contexto, los *Prompts* con mayor índice de éxito son aquellos que aglutinan palabras y formas de redacción con mayor similitud a los abstracts.
3. **Eficiencia Computacional:** El tiempo promedio de respuesta fue de alrededor de **1 minuto**, lo cual sugiere que el sistema puede beneficiarse de optimización en la GPU (teniendo en cuenta las limitaciones del entorno de *Google Collab*. Los tiempos de ejecución son los siguientes:
 - **Tiempo de Recopilación de Abstracts:** 10 segundos.
 - **Tiempo de Chunking y Embedding:** 8 minutos.
 - **Tiempo de asimilación de pregunta y generación de respuesta:** 1 minuto.

La parte con mayor carga computacional es la parte de *Chunking* y *Embedding* por tanto una vez establecido el conocimiento, las consultas son medianamente ágiles.

4. Conclusiones

En conclusión, los resultados demuestran que el sistema RAG implementado exhibe una notable eficacia en la fase de recuperación de información, validada mediante la proyección espacial UMAP y la métrica de similitud coseno, asegurando que el contexto proporcionado al modelo sea semánticamente relevante. No obstante, el análisis de la fase de generación revela que el LLM opera principalmente como un motor de extracción literal y síntesis fragmentaria, mostrando limitaciones en la creación de lenguaje original o en la comprensión profunda de conceptos físicos complejos. A pesar de estas restricciones en la capacidad de razonamiento autónomo, la alta fidelidad en la recuperación de datos cuantitativos y la robustez demostrada confirman que el pipeline es una herramienta medianamente funcional para la consulta automatizada de literatura científica, cuya agilidad operativa se ve optimizada una vez consolidada la base de conocimientos vectorial.